

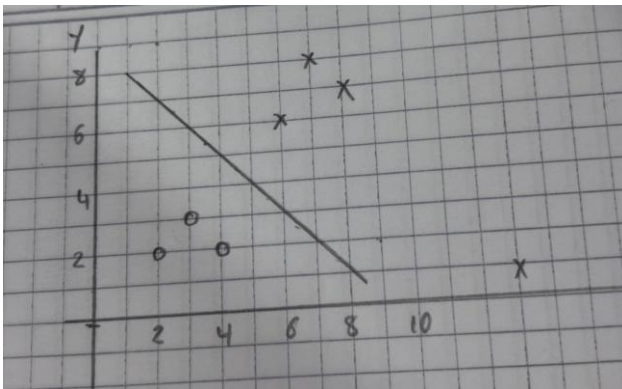
TALLER ANALÍTICO: DIBUJANDO EL MARGEN (20 MIN)

Dibujen en su cuaderno un plano cartesiano (ejes X e Y de 0 a 10). Ubiquen los siguientes puntos:

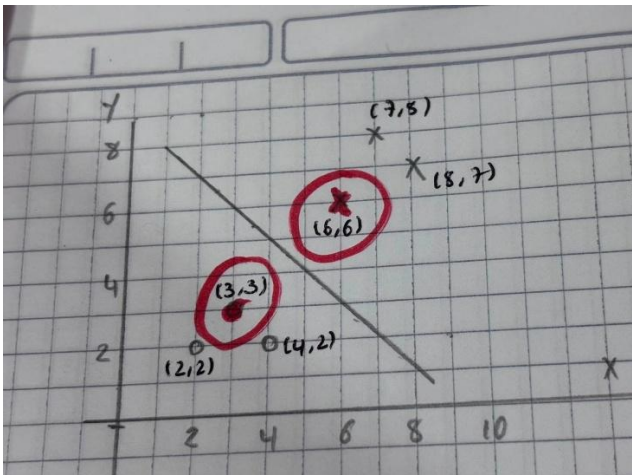
- Clase A (Círculos): (2,2), (3,3), (4,2).
- Clase B (Equis): (6,6), (7,8), (8,7).

1. Tracen con regla la línea recta óptima que separe ambos grupos buscando dejar el mismo espacio de "calle" a ambos lados.
2. Marquen con un círculo rojo cuáles de esos 6 puntos actuarían como los "Vectores de Soporte" de su modelo.
3. Si agregamos un nuevo punto de Clase A en la coordenada (1,1), ¿cambiaría la posición de la línea? Justifiquen por qué sí o por qué no basándose en la teoría.

1.



2.



3. No, la línea no se movería en absoluto.

La razón es simple: el punto (1,1) está aún más alejado del grupo B que todos los demás puntos de la Clase A. Está en la esquina de abajo a la izquierda, lejísimos de la frontera.

Como vimos en la teoría, al algoritmo SVM solo le importan los puntos que están en el borde del margen (los Vectores de Soporte). Los puntos que están bien adentro de su propio territorio, lejos de la línea, son completamente ignorados por el algoritmo, porque no aportan ninguna información nueva sobre dónde está el límite entre los dos grupos. Es como si para decidir dónde poner la cerca entre dos casas vecinas, solo importara dónde termina el jardín de cada una, no dónde está la cocina.

TALLER DE LABORATORIO: FRONTERAS NO LINEALES (40 MIN)

Misión Práctica:

1. Transcriban y ejecuten el código superior. Observen los vectores de soporte impresos. ¿Coinciden con los que marcaron en su cuaderno durante el taller analítico?
2. Ahora, vamos a "engañar" a la frontera lineal. Agreguen un nuevo punto a la variable X: [5, 5] y asígñenle la etiqueta 0 (Clase A) en la variable Y.
3. Vuelvan a entrenar el modelo lineal. Intentará trazar una recta pero probablemente fallará o se verá forzado.
4. Cambien el hiperparámetro de `kernel='linear'` a `kernel='rbf'`. Vuelvan a entrenar y a predecir.
5. **Reflexión:** El "Kernel Trick" (RBF) le permite al SVM aislar puntos que están "rodeados" por la clase enemiga. ¿En qué escenario del mundo real (ej. medicina o reconocimiento facial) creen que un kernel lineal fallaría completamente y se requeriría RBF?

```

Problems Output Debug Console Terminal Ports 1
@gozzuley19-debug → /workspaces/IA-2-Talleres (main) $ source /workspaces/IA-2-Talleres/
(.venv) @gozzuley19-debug → /workspaces/IA-2-Talleres (main) $ /workspaces/IA-2-Talleres/
===== PARTE 1: MODELO LINEAL ORIGINAL =====

Los Vectores de Soporte son:
[[3. 3.]
 [4. 2.]
 [6. 6.]]
El punto [5,4] pertenece a la clase: 0

===== PARTE 2: AGREGANDO EL PUNTO 'PROBLEMÁTICO' [5,5] =====

--- Modelo LINEAL con el nuevo punto ---
Vectores de Soporte:
[[5. 5.]
 [6. 6.]]
Precisión del modelo lineal: 1.0
Predicción para [5,5]: 0 (debería ser 0)

===== PARTE 3: CAMBIANDO A KERNEL RBF =====

--- Modelo RBF con el nuevo punto ---
Vectores de Soporte:
[[2. 2.]
 [4. 2.]
 [5. 5.]
 [6. 6.]
 [7. 8.]
 [8. 7.]]
Precisión del modelo RBF: 1.0
Predicción para [5,5]: 0 (debería ser 0)
(.venv) @gozzuley19-debug → /workspaces/IA-2-Talleres (main) $

```

Un kernel lineal puede tener problemas en algunos casos de diagnóstico médico, por ejemplo, para saber si un tumor es benigno o maligno según su tamaño y densidad. Esto pasa porque los tumores malignos no siempre tienen valores parecidos. Algunos pueden ser pequeños y otros grandes, por lo que pueden quedar mezclados con los tumores benignos. Una línea recta no podría separar correctamente estos casos.

Algo parecido ocurre con el reconocimiento facial. Una misma persona puede verse diferente dependiendo de la luz, el ángulo de la cámara o si usa gafas. Por eso, sus datos pueden aparecer en diferentes partes del gráfico y mezclarse con los de otras personas. El kernel RBF puede crear límites curvos para separar mejor estos grupos, mientras que una línea recta tendría más dificultad.

En resumen, el kernel RBF es útil cuando los datos están mezclados o no se pueden separar usando una simple línea recta.